

CIS 3200 – Homework 6

Sunny Lee
lseonu@seas.upenn.edu

July 17, 2026

Partners: Yeongheon Lee, Josh Lee.

Problem 1

Reduction Method: We will prove that this question of monotone satisfiability with fewest true variables is NP-hard through the reduction from the vertex cover problem. First, note that in class, we learned that the input of a vertex-cover problem is a graph $G = (V, E)$ and k and the goal is to output a vertex-cover of smallest size (S is a vertex cover of G if $\forall (u, v) \in E$, we have $|S \cap \{u, v\}| \geq 1$). We will reduce the vertex-cover problem to the given problem. The input would be $G = (V, E)$ and an integer $k' = k$. Then, to construct the monotone SAT as described in the problem, for each vertex in V , we will initialize a new variable a_i , $1 \leq i \leq n$ that corresponds to the vertex v_i . Then, we set a_i to True if v_i is in the vertex cover, and for each edge $(v_i, v_j) \in E$, we will make a clause, $a_i \vee a_j$ so that at least one vertex is in the vertex-cover. Then, the monotone SAT can have a satisfying assignment made of at most k' number of True variables if and only if the original graph in the vertex cover problem has a vertex cover of size at most k' .

Proof of Correctness:

First, to prove Vertex cover problem $\Rightarrow$ given problem, we know that in the algorithm, suppose S is a vertex cover of size at most k . That is, S must have at least one end of all edges must be in set S . Then, the variable x_i must be set to True if the vertex v_i is in S . It would be set to false in every other situation. Now, we also know that each of the clause $x_i \vee x_j$ represent each edge (v_i, v_j) . Because at least one of v_i or v_j would be in the set S , we know that this makes the clause satisfied. Therefore, we know that at most $k = k'$ variables will be set as true.

Now, to prove the opposite direction, $\Leftarrow$, suppose that the SAT is satisfied with at most $k=k'$ variables set to true. Then, that means that $S = \{v_i | x_i = \text{true}\}$. That is, for each of the clauses $x_i \vee x_j$, either v_i , v_j , or both of them must be in set S since at least one of x_i or x_j must be set to true. Therefore, we know that S must be a vertex cover of size at most k .

We know from lecture that vertex cover problem is NP-hard. Therefore, this reduction indicates the NP-hardness of the given problem as well.

Running Time:

We need to create variables and clauses. Creating variables take $O(|V|)$ time and creating clauses take $O(|E|)$ time. Therefore, the total runtime is $O(|V| + |E|)$

Problem 2

Reduction Method:

We will reduce the Hamiltonian cycle(Input: $G(V,E)$) problem to the spanning tree packing problem(Input: $G'(V',E')$, int k , and int l) by setting:

- $V' = V$
- $E' = E$
- $G'(V', E') = G(V, E)$
- $k = |V|$
- $l = |V|$
- Graph $G(V, E)$ contains a Hamiltonian cycle if and only if G' has a subset F that satisfies:
 - $F \subseteq E$
 - $|F| \leq |V|$

and G' must have a minimum of $|V|$ unique spanning trees consisting of only edges in set F .

Proof of Correctness:

$\Rightarrow$: To prove this direction, we suppose that $G(V, E)$ contains a unique Hamiltonian cycle, H . Then, suppose F is the set containing all edges in Hamiltonian cycle H . Then, we know that since H is a Hamiltonian cycle, by definition, it must visit all $|V|$ vertices exactly once. Thus, $|F| = |V|$. That is, for each edge (u, v) in F , removing it from the cycle H would create a distinct spanning tree. This spanning tree created would also be a Hamiltonian path. Now, we know that H has $|V|$ number of edges, and therefore could yield n unique spanning trees with each unique edge removed. That is, $|F| = |V|$, $|F| \leq k$, and $|V| \leq l$. Therefore, the spanning tree packing problem is satisfied.

$\Leftarrow$: Now, to prove the opposite direction, we suppose that there is a subset F that satisfies: $F \subseteq E$, $|F| \leq |V|$, and there are at least $|V|$ distinct spanning trees of G from edges in F . Note how $|F| \geq |V|$ since to have $|V|$ unique trees, there must be $|V|$ edges since each tree would differ by at least a single edge and each spanning tree must have $|V| - 1$ edges. That is, $|F| = |V|$, and since we need each of the spanning tree to be formed by removing only one edge from set F , we know that F must be a cycle and for the tree to be a spanning tree, F must have been a cycle that spans all vertices with a cardinality of $|V|$. Therefore, we know that F must be a Hamiltonian cycle and thus G must contain a Hamiltonian cycle.

Now, we know that the Hamiltonian cycle problem is a NP-complete problem and did a poly-time reduction. Therefore, the given problem is NP-hard.

Running Time:

We are simply copying $G(V,E)$, which takes $O(|V| + |E|)$ time and setting integers k and l , which takes $O(1)$ time, thus takes $O(|V| + |E|)$ time.

Problem 3

Reduction Method: We will reduce the 3-SAT problem to the given problem. For each of the variables x_i , we will construct one column that is red if x_i is true, else empty, and another column that is the negation of x_i , thus is blue if x_i is false, else empty. Then for each clause C_j , we will construct a row that is red if the literal is x_i and blue if it is the negation of x_i , else empty. Now, the 3-SAT is satisfied if and only if there are no columns with both blue and red stones (must ensure no column has both colors at any position) and each of the row has at least one stone.

Proof of Correctness:

$\Rightarrow$: Note how if x_i is true, column x_i must be red and the column of negation of x_i must be empty. Similarly, if x_i is false, column of negation of x_i is blue and the column of x_i is empty. This construction ensures that there is only one color for each column or else it is empty. This prevents any column from having both colors) Moreover, we also know that each row must be satisfied since there must be at least one stone in each row (clause) since at least one of the literals must be true.

$\Leftarrow$: Now, for the other direction, we know that a valid solution for the puzzle problem ensures that for each variable x_i , x_i would be true if there is a red stone in the column of x_i and false if there is a blue stone in the column of $\neg x_i$. Since there cannot exist columns with both colors (at any position), this ensures that there cannot be any variables with both boolean values and each row would have a minimum of one stone, thus all of the clauses would be satisfied.

NP Completeness: First of all, we know that the problem is NP because this reduction can be verified in polynomial time. We simply check if no columns have both colored stones, which takes $O(mn)$ time, and also that each row has at least one stone of any color, which also takes $O(mn)$ time. Now, we showed that we can do a polynomial time reduction from the 3SAT problem, which we learned from lecture to be NP-complete, to the given problem. Therefore, this shows that the problem is NP hard as well. Because we proved that the problem is both in NP and NP hard, this is NP complete.

Running Time: There are a total of $2n$ columns and m rows for n variables and m clauses. That is, we set stones for $m \times 2n$ grid, therefore it takes $O(mn)$ time and thus it is polynomial time.

Problem 4**Algorithm:**

- **Step 1:** Partition the loaves into set H and L, H being the set of loaves weighing at least M ounces and L being the set of loaves weighing less than M ounces. Put each loaf in set H in a separate box and keep track using variable b_H .
- **Step 2:** Using merge sort, sort all loaves in set L so that they are in a non-decreasing order of weight.
- **Step 3:** Set two pointers, r and l, which is set to $r = |L|-1$ and $l = 0$, as well as a variable for number of boxes in set L, $b_L = 0$.
- **Step 4:** While $l \leq r$, check if $W[l] + W[r] \geq M$. If so, we put those two loaves in a single box and increment l and b_L , decrement r, and start a new box. Else, increment l as the current loaf at l cannot be paired with any remaining loaf to reach M ounces.
- **Step 5:** Return the boxes = $b_H + b_L$.

Proof of Correctness:

Because of the condition in step 4, we ensure that all of the packed boxes must be heavier than M ounces. Moreover, all boxes made in step 1 must also be heavier than M ounces as well. Now, suppose OPT indicates the optimal number of boxes that can be created such that each box is heavier than M ounces. We know that for set H, $b_H = |H|$ must be optimal since each loaf in set H must have its own box to maximize number of boxes. Then, for set L, let w_{total} be the total weight of loaves in L. Then, $OPT \leq |H| + \lceil \frac{w_{total}}{M} \rceil$ since each box needs at least M ounces. In our algorithm, each successful pair results in a box with $\geq M$ ounces and at most one loaf (lightest) is unpaired. Thus, $b = b_H + b_L \geq |H| + \frac{1}{2} \lceil \frac{w_{total}}{M} \rceil \geq \frac{1}{2} OPT$ and therefore this proves 2-approximation.

Running Time: Partitioning takes $O(n)$ time, merge sort on set L takes $O(|H| \log |H|)$, which is less than or equal to $O(n \log n)$. The tracking pointer step takes $O(n)$ time. Thus, the total runtime is $O(n \log n)$ time thus is polynomial.

Problem 5

a) Note that the largest task has to be processed on any machine, including the optimal one in any schedule. That is, there cannot be a machine that finishes all tasks before the longest one, thus $T^* \geq \max_{1 \leq i \leq n} p_i$ holds true.

b) The total time for processing, $\sum_{i=1}^n p_i$, is going to be split between m number of machines. That is, the average load is $\sum_{i=1}^n p_i$ divided by m , thus there must exist at least one machine with at least the average load. Therefore, $T^* \geq \frac{1}{m} \sum_{i=1}^n p_i$

c) There must be a machine (set this to machine l) that finishes last, and thus reaches the total time needed to complete all tasks (T). Then, before machine l starts the last task, according to the greedy algorithm, the machine must have had the smallest load. This indicates that $T - p_l \leq \frac{1}{m} \sum_{i \neq l} p_i$ since machine l had minimal load before last task. Now, rearranging, we get $T \leq p_l + \frac{1}{m} \sum_{i \neq l} p_i$. Then, based on (a) and (b), we know that $\max p_i + \frac{1}{m} \sum p_i \leq T^* + T^* = 2T^*$. Since T is at most $2T^*$, this proves 2-approximation.